COMPUTER NETWORKS

Week 7

Socket Communication: Definitions, Classification and Java Client-Server Programming over TCP (Transmission Control Protocol) and UDP (User Datagram Protocol)

Lecture • Seminar • Laboratory (Windows and Linux tracks)

Contents

Learning objectives and link with Week 6

Part I — Lecture

1. From transport services to applications
2. Sockets: definitions
3. Classification of sockets
4. The socket API (Application Programming Interface) and call sequences
5. Network programming in Java
6. TCP (Transmission Control Protocol) client-server communication in Java
7. UDP (User Datagram Protocol) client-server communication in Java
8. TCP (Transmission Control Protocol) vs UDP (User Datagram Protocol) programming; good practices
9. Summary and key terms

Part II — Seminar: analysis, debugging and protocol design

Part III — Laboratory

Lab 7-W: Java sockets on Windows

Lab 7-L: Java sockets on Linux

Programming assignments (both tracks)

Appendix — reference solution and references

Learning objectives and link with Week 6

In Week 6 we studied the transport layer from the network's point of view: ports, UDP (User Datagram Protocol) and TCP (Transmission Control Protocol) headers, the three-way handshake, sliding windows and congestion control. **This week (Week 7)** looks at the same layer from the **programmer's point of view**. Applications never build TCP (Transmission Control Protocol) segments themselves; they ask the operating system for a **socket** and read and write data through it. We define sockets, classify them, study the socket API (Application Programming Interface) and write complete TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) client-server programs in **Java**, which we then build and test on **Windows** and **Linux**.

By the end of this week, students should be able to:

1. Define a socket, a socket address and a connection, and relate them to the 5-tuple from Week 6.
2. Classify sockets by address family, type, role, I/O (Input/Output) model and delivery mode.
3. Describe the order of socket API (Application Programming Interface) calls for TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) clients and servers and relate each call to the packets it causes.
4. Use the Java classes `InetAddress`, `Socket`, `ServerSocket`, `DatagramSocket` and `DatagramPacket`.
5. Write, compile and run iterative and concurrent TCP (Transmission Control Protocol) servers and UDP (User Datagram Protocol) servers with their clients.
6. Design a simple text application protocol and handle message framing over a TCP (Transmission Control Protocol) byte stream.
7. Diagnose typical socket errors (connection refused, address in use, timeouts, firewall blocks) on Windows and Linux.

Part I — Lecture

1. From transport services to applications

Network applications are built from processes that exchange messages. The most common organisation is the **client-server model**: a **server** process runs permanently at a known address and port, waits for requests and answers them; a **client** process starts the communication when it needs a service. In the **peer-to-peer** model, each process acts as both client and server. In both models, the processes access the transport layer through the same interface: the **socket API (Application Programming Interface)**, provided by the operating system.

Table 1. *Roles in the client-server model*

Characteristic	Server	Client
Address and port	Fixed, well known (e.g. port 80, 443, 5000)	Usually any; the OS (Operating System) assigns an ephemeral port
Life time	Runs continuously, waiting for requests	Runs when the user needs the service
Initiative	Passive: waits (listen / receive)	Active: connects / sends first
Concurrency	Serves many clients (iteratively or concurrently)	Usually talks to one server at a time

2. Sockets: definitions

> **Definition**
>
> A **socket** is an endpoint of a bidirectional communication channel between two processes, created and managed by the operating system. The application identifies it by a **handle** (a *file descriptor* on Linux/Unix, a *SOCKET* handle on Windows, a `Socket` object in Java); the network identifies it by its **socket address** (IP (Internet Protocol) address, port) and transport protocol.

- **Socket address** — the pair (IP (Internet Protocol) address, port), e.g. `192.168.1.10:5000`. In Java: `InetSocketAddress`.
- **Connection** — for TCP (Transmission Control Protocol), the association of two socket addresses, identified by the 5-tuple (protocol, local IP (Internet Protocol), local port, remote IP (Internet Protocol), remote port).
- **Listening (passive) socket** — a server TCP (Transmission Control Protocol) socket that only accepts connection requests; it never carries data.
- **Connected socket** — a TCP (Transmission Control Protocol) socket bound to one specific connection; it carries the data. A server has one listening socket plus one connected socket per client.
- **Socket API (Application Programming Interface)** — the set of system calls used to create and use sockets. It originated in **4.2BSD (4.2 Berkeley Software Distribution) Unix (1983)** as *Berkeley sockets*, was standardised in **POSIX (Portable Operating System Interface)**, and was adopted on Windows as **Winsock** (Windows Sockets, 1992). Java's `java.net` package (since JDK (Java Development Kit) 1.0, 1996) wraps the same API (Application Programming Interface) in classes.

A useful image: if the host is a building and the IP (Internet Protocol) address is its street address, ports are the numbered mailboxes in the lobby and a socket is the specific door through which a process sends and receives its mail. Everything above the socket belongs to the application; everything below it (TCP (Transmission Control Protocol)/UDP (User Datagram Protocol), IP (Internet Protocol), the driver) belongs to the operating system.

3. Classification of sockets

Table 2. *Classification of sockets*

Criterion	Classes	Notes / examples
Address family (communication domain)	AF_INET (IPv4 (Internet Protocol version 4)) AF_INET6 (IPv6 (Internet Protocol version 6)) AF_UNIX / AF_LOCAL AF_PACKET, AF_BLUETOOTH, …	IPv4 (Internet Protocol version 4) and IPv6 (Internet Protocol version 6) are used for network communication; AF_UNIX connects processes on the same host through a file-system path (supported on Linux and on Windows 10 1803+); AF_PACKET (Linux) gives access to link-layer frames
Socket type (service)	SOCK_STREAM SOCK_DGRAM SOCK_RAW SOCK_SEQPACKET	Stream → TCP (Transmission Control Protocol): reliable, ordered byte stream. Datagram → UDP (User Datagram Protocol): independent messages. Raw → direct access to IP (Internet Protocol) (ping, traceroute, packet tools; needs administrator rights). Seqpacket → reliable messages (SCTP (Stream Control Transmission Protocol))
Connection	Connection-oriented Connectionless	TCP (Transmission Control Protocol) sockets must connect before transferring data; UDP (User Datagram Protocol) sockets can send to any address at any time
Role	Server (passive) Client (active)	Server: socket → bind → listen → accept. Client: socket → connect
I/O (Input/Output) model	Blocking Non-blocking Multiplexed Asynchronous	Blocking calls wait (default in java.net). Non-blocking calls return immediately. Multiplexing: select/poll/epoll (Linux), IOCP (Input/Output Completion Ports) (Windows), Java NIO (New Input/Output) `Selector`. Asynchronous: completion callbacks (Java `AsynchronousSocketChannel`)
Delivery mode	Unicast Broadcast Multicast	Broadcast and multicast are possible only with datagram sockets (Java `DatagramSocket` with broadcast enabled, `MulticastSocket`)

In this course we use **Internet sockets** (AF_INET / AF_INET6) of the two main types: **stream sockets (TCP (Transmission Control Protocol))** and **datagram sockets (UDP (User Datagram Protocol))**, in **blocking** mode, which is the simplest model to reason about.

4. The socket API (Application Programming Interface) and call sequences

Table 3. *Berkeley socket primitives*

Primitive (C / POSIX (Portable Operating System Interface))	Used by	Purpose	Network effect
socket()	TCP (Transmission Control Protocol), UDP (User Datagram Protocol)	Create a socket of a given family and type; returns a handle	None
bind()	Servers (and optionally clients)	Attach a local IP (Internet Protocol) address and port	None
listen()	TCP (Transmission Control Protocol) server	Mark the socket as passive; create the queue of pending connections (backlog)	Port appears in state LISTEN
accept()	TCP (Transmission Control Protocol) server	Take a completed connection from the queue; returns a new connected socket	Handshake was completed by the kernel

Primitive (C / POSIX (Portable Operating System Interface))	Used by	Purpose	Network effect
connect()	TCP (Transmission Control Protocol) client (UDP (User Datagram Protocol) optional)	Open a connection to a remote socket address	TCP (Transmission Control Protocol): sends SYN (Synchronize), waits for SYN-ACK (Synchronize-Acknowledgment), sends ACK (Acknowledgment)
send() / recv() write() / read()	TCP (Transmission Control Protocol) (connected UDP (User Datagram Protocol))	Send / receive bytes on a connected socket	Data segments and ACKs (Acknowledgments)
sendto() / recvfrom()	UDP (User Datagram Protocol)	Send / receive one datagram to / from an explicit address	One UDP (User Datagram Protocol) datagram per call
shutdown()	TCP (Transmission Control Protocol)	Close one direction (half-close)	Sends FIN (Finish)
close() / closesocket()	TCP (Transmission Control Protocol), UDP (User Datagram Protocol)	Release the socket	TCP (Transmission Control Protocol): FIN (Finish) (or RST (Reset) if unread data remains)

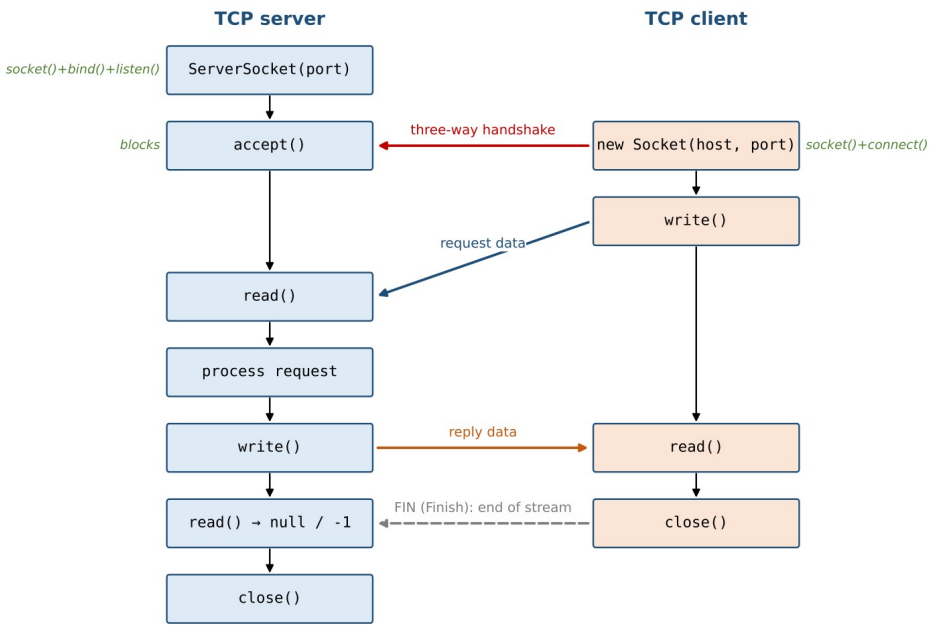


Figure 1. *Sequence of socket calls for a TCP (Transmission Control Protocol) client and server (Java method names, with the corresponding POSIX (Portable Operating System Interface) calls in italics)*

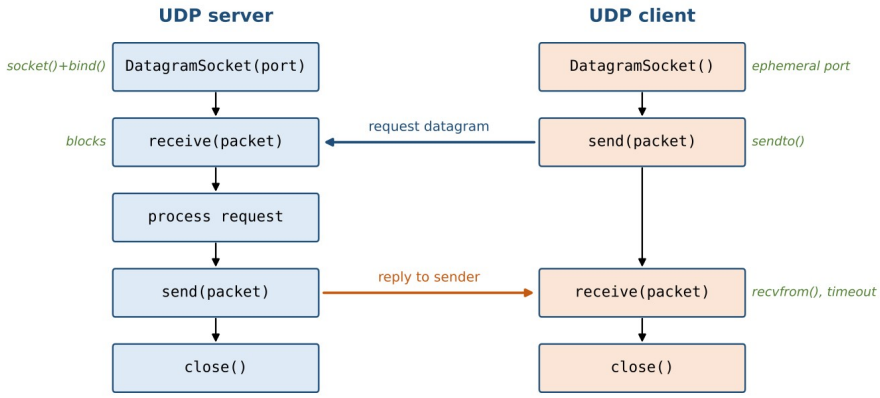


Figure 2. *Sequence of socket calls for a UDP (User Datagram Protocol) client and server — there is no connection set-up*

5. Network programming in Java

Java offers network programming at several levels. The package **java.net** contains the classic blocking socket classes used in this course; **java.nio.channels** adds non-blocking and multiplexed I/O (Input/Output); higher-level classes such as `java.net.http.HttpClient` implement complete application protocols.

Table 4. *Main classes of java.net for socket programming*

Class	Purpose	Main methods
InetAddress	An IP (Internet Protocol) address (IPv4 (Internet Protocol version 4) or IPv6 (Internet Protocol version 6)), with name resolution	getByName(), getAllByName(), getLocalHost(), getLoopbackAddress(), getHostAddress(), getHostName()
InetSocketAddress	IP (Internet Protocol) address + port	new InetSocketAddress(host, port), getAddress(), getPort()
ServerSocket	TCP (Transmission Control Protocol) listening socket (socket + bind + listen)	ServerSocket(port[, backlog[, bindAddr]]), accept(), close(), setSoTimeout()
Socket	TCP (Transmission Control Protocol) connected socket (client side or accepted on the server)	Socket(host, port), getInputStream(), getOutputStream(), shutdownOutput(), setSoTimeout(), setTcpNoDelay(), close()
DatagramSocket	UDP (User Datagram Protocol) socket	DatagramSocket([port]), send(), receive(), connect(), setSoTimeout(), setBroadcast(), close()
DatagramPacket	One UDP (User Datagram Protocol) datagram: data buffer + length + remote address/port	getData(), getLength(), getAddress(), getPort(), setLength()
MulticastSocket	UDP (User Datagram Protocol) socket that can join multicast groups	joinGroup(), leaveGroup()

Table 5. *Common networking exceptions in Java*

Exception	Typical cause
UnknownHostException	The host name cannot be resolved (DNS (Domain Name System) error, typing mistake)
ConnectException: Connection refused	No process listens on the destination port: the server host answered with RST (Reset)
SocketTimeoutException	A blocking call (connect, accept, read, receive) exceeded the timeout set with setSoTimeout()

Exception	Typical cause
BindException: Address already in use	Another socket (or a previous run of the same program) already uses the port
BindException: Permission denied	On Linux, binding to a port < 1024 without root rights
SocketException: Connection reset	The peer aborted the connection (RST (Reset)), e.g. it was killed
PortUnreachableException	A connected DatagramSocket received an ICMP (Internet Control Message Protocol) Port Unreachable

Listing 1. Working with InetAddress

```
import java.net.*;

public class AddressInfo {
    public static void main(String[] args) throws Exception {
        String name = (args.length > 0) ? args[0] : "www.example.com";
        for (InetAddress a : InetAddress.getAllByName(name))     // DNS (Domain Name System) resolution (A
and AAAA records)
            System.out.println(name + " -> " + a.getHostAddress());
        System.out.println("Local host: " + InetAddress.getLocalHost());
        System.out.println("Loopback  : " + InetAddress.getLoopbackAddress());
    }
}
```

6. TCP (Transmission Control Protocol) client-server communication in Java

6.1 Steps

Table 6. *Steps of a TCP (Transmission Control Protocol) server and client*

TCP (Transmission Control Protocol) server	TCP (Transmission Control Protocol) client
1. Create a `ServerSocket` on a known port (socket + bind + listen).	1. Create a `Socket(host, port)` — this performs the three-way handshake.
2. Call `accept()`; it blocks until a client connects and returns a connected `Socket`.	2. Obtain input and output streams from the socket.
3. Obtain input/output streams from the connected socket.	3. Send a request, read the reply, according to the application protocol.
4. Read requests, process them, write replies.	4. Close the socket (sends FIN (Finish)).
5. Close the connected socket; go back to step 2 for the next client.	

6.2 A concurrent TCP (Transmission Control Protocol) echo server

The server below listens on a port given on the command line (default 5000). For every accepted client it starts a new thread, so several clients can be served at the same time. The *application protocol* is line based: the client sends a line of text; the server replies with `ECHO:` followed by the line in upper case; the line `BYE` ends the session.

Listing 2. TcpEchoServer.java

```
import java.io.*;
import java.net.*;

public class TcpEchoServer {
    public static void main(String[] args) throws IOException {
        int port = (args.length > 0) ? Integer.parseInt(args[0]) : 5000;
        try (ServerSocket server = new ServerSocket(port)) {
            System.out.println("Echo server (stream socket) listening on port " + port);
            while (true) {
                Socket client = server.accept();          // blocks until a client connects
```

```
                new Thread(() -> handle(client)).start(); // one thread per client
            }
        }
    }

    private static void handle(Socket s) {
        String who = s.getInetAddress().getHostAddress() + ":" + s.getPort();
        System.out.println("Connected: " + who);
        try (s;
             BufferedReader in = new BufferedReader(
                     new InputStreamReader(s.getInputStream(), "UTF-8"));
             PrintWriter out = new PrintWriter(
                     new OutputStreamWriter(s.getOutputStream(), "UTF-8"), true)) {
            String line;
            while ((line = in.readLine()) != null) {
                if (line.equalsIgnoreCase("BYE")) { out.println("Goodbye"); break; }
                out.println("ECHO: " + line.toUpperCase());
            }
        } catch (IOException e) {
            System.out.println("Error with " + who + ": " + e.getMessage());
        }
        System.out.println("Disconnected: " + who);
    }
}
```

Listing 3. TcpEchoClient.java

```
import java.io.*;
import java.net.*;

public class TcpEchoClient {
    public static void main(String[] args) throws IOException {
        String host = (args.length > 0) ? args[0] : "localhost";
        int port = (args.length > 1) ? Integer.parseInt(args[1]) : 5000;
        try (Socket s = new Socket(host, port);   // three-way handshake happens here
             BufferedReader in = new BufferedReader(
                     new InputStreamReader(s.getInputStream(), "UTF-8"));
             PrintWriter out = new PrintWriter(
                     new OutputStreamWriter(s.getOutputStream(), "UTF-8"), true);
             BufferedReader kb = new BufferedReader(new InputStreamReader(System.in))) {
            System.out.println("Connected to " + host + ":" + port
                               + " from local port " + s.getLocalPort());
            String line;
            while ((line = kb.readLine()) != null) {
                out.println(line);
                String reply = in.readLine();
                if (reply == null) break;
                System.out.println(reply);
                if (line.equalsIgnoreCase("BYE")) break;
            }
        }
    }
}
```

6.3 Explanation

- `new ServerSocket(port)` creates the listening socket; the port appears as LISTEN in `netstat` / `ss`. The backlog (default 50) is the length of the queue of completed connections waiting for `accept()`.
- `accept()` blocks; when it returns, the handshake is already complete. The returned socket has the same local port as the server socket — it is distinguished by the client's address and port.
- `new Socket(host, port)` resolves the name, chooses an ephemeral local port and performs the handshake. If no server listens, it throws `ConnectException`.
- TCP (Transmission Control Protocol) is a **byte stream**. `BufferedReader.readLine()` and `PrintWriter.println()` impose a *line-based* framing on top of it: every message ends with a newline.
- `PrintWriter(..., true)` enables **auto-flush** on `println()`. Without flushing, data stays in the buffer and the peer waits forever — a classic bug.
- `readLine()` returns `null` when the peer closes the connection (end of stream, FIN (Finish) received).

- **try-with-resources** closes streams and sockets automatically, even after an exception. Always specify a charset (UTF-8 (8-bit Unicode Transformation Format)) so that Windows and Linux machines interoperate.

6.4 Iterative vs concurrent servers

Table 7. *Server concurrency models*

Model	How it works	Advantages	Drawbacks
Iterative	accept → serve the client completely → accept the next	Very simple; no shared state	One slow client blocks all others
Thread per client	accept → start a new thread for the client (Listing 2)	Simple; clients served in parallel	Thousands of clients mean thousands of platform threads
Thread pool	accept → submit the client to an ExecutorService	Bounded resource use	Pool size limits the number of simultaneous clients
Virtual threads (Java 21+)	accept → one cheap virtual thread per client	Simple code, scales to very many clients	Requires a recent JDK (Java Development Kit)
Event-driven (NIO (New Input/Output))	One thread multiplexes many non-blocking channels with a Selector	Highest scalability	More complex code

Listing 4. Thread-pool variant of the accept loop

```
ExecutorService pool = Executors.newFixedThreadPool(50);   // or Executors.newVirtualThreadPerTaskExecutor()
(Java 21+)
try (ServerSocket server = new ServerSocket(5000)) {
    while (true) {
        Socket client = server.accept();
        pool.submit(() -> handle(client));
    }
}
```

6.5 Message framing over TCP (Transmission Control Protocol)

Because TCP (Transmission Control Protocol) does not preserve message boundaries, the application must define where a message ends. Three common techniques are: a **delimiter** (e.g. newline, as in HTTP (Hypertext Transfer Protocol) headers, SMTP (Simple Mail Transfer Protocol), our echo protocol), a **length prefix** (binary protocols), or **fixed-size** messages. A length-prefixed exchange in Java:

Listing 5. Length-prefix framing

```
// sender
DataOutputStream out = new DataOutputStream(new BufferedOutputStream(socket.getOutputStream()));
byte[] msg = text.getBytes(StandardCharsets.UTF_8);
out.writeInt(msg.length);   // 4-byte length, big-endian (network byte order)
out.write(msg);
out.flush();


// receiver
DataInputStream in = new DataInputStream(new BufferedInputStream(socket.getInputStream()));
int len = in.readInt();
byte[] buf = new byte[len];
in.readFully(buf);          // loops until exactly len bytes have arrived
String text = new String(buf, StandardCharsets.UTF_8);
```

6.6 Useful socket options

Table 8. *TCP (Transmission Control Protocol) socket options*

Option (Java method)	Effect
setSoTimeout(ms)	Maximum blocking time of read()/accept()/receive(); then SocketTimeoutException

Option (Java method)	Effect
connect(addr, timeoutMs)	Limit the time spent on the handshake (use `new Socket()` then `connect()`)
setTcpNoDelay(true)	Disable Nagle's algorithm: small writes are sent immediately (interactive protocols, games)
setKeepAlive(true)	Send periodic keep-alive probes on idle connections to detect dead peers
setReuseAddress(true)	Allow a server to bind its port again while old connections are in TIME_WAIT
setReceiveBufferSize / setSendBufferSize	Change kernel buffer sizes — influences the advertised receive window
shutdownOutput()	Half-close: send FIN (Finish) but keep reading the peer's answer

7. UDP (User Datagram Protocol) client-server communication in Java

7.1 Steps

Table 9. *Steps of a UDP (User Datagram Protocol) server and client*

UDP (User Datagram Protocol) server	UDP (User Datagram Protocol) client
1. Create a `DatagramSocket(port)` on a known port.	1. Create a `DatagramSocket()` — the OS (Operating System) chooses an ephemeral port.
2. Prepare an empty `DatagramPacket` with a buffer and call `receive()` — it blocks until a datagram arrives.	2. Build a `DatagramPacket` with the data, the server address and port; call `send()`.
3. Read the data (`getData()`, `getLength()`) and the sender's address (`getAddress()`, `getPort()`).	3. Call `receive()` for the reply, with a timeout (`setSoTimeout`), because the request or the reply may be lost.
4. Build a reply packet addressed to the sender and `send()` it; loop to step 2.	4. Close the socket.

Listing 6. UdpEchoServer.java

```
import java.net.*;
import java.nio.charset.StandardCharsets;

public class UdpEchoServer {
    public static void main(String[] args) throws Exception {
        int port = (args.length > 0) ? Integer.parseInt(args[0]) : 6000;
        try (DatagramSocket socket = new DatagramSocket(port)) {
            System.out.println("Echo server (datagram socket) listening on port " + port);
            byte[] buf = new byte[1024];
            while (true) {
                DatagramPacket req = new DatagramPacket(buf, buf.length);
                socket.receive(req);                        // blocks until a datagram arrives
                String msg = new String(req.getData(), 0, req.getLength(), StandardCharsets.UTF_8);
                System.out.println("From " + req.getAddress().getHostAddress() + ":"
                                   + req.getPort() + " -> " + msg);
                byte[] ans = ("ECHO: " + msg.toUpperCase()).getBytes(StandardCharsets.UTF_8);
                socket.send(new DatagramPacket(ans, ans.length, req.getAddress(), req.getPort()));
            }
        }
    }
}
```

Listing 7. UdpEchoClient.java

```
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;

public class UdpEchoClient {
    public static void main(String[] args) throws Exception {
        InetAddress server = InetAddress.getByName((args.length > 0) ? args[0] : "localhost");
```

```
        int port = (args.length > 1) ? Integer.parseInt(args[1]) : 6000;
        try (DatagramSocket socket = new DatagramSocket();   // ephemeral local port, no connection
             BufferedReader kb = new BufferedReader(new InputStreamReader(System.in))) {
            socket.setSoTimeout(2000);                       // UDP (User Datagram Protocol) gives no
guarantee: use a timeout
            String line;
            while ((line = kb.readLine()) != null && !line.equalsIgnoreCase("BYE")) {
                byte[] data = line.getBytes(StandardCharsets.UTF_8);
                socket.send(new DatagramPacket(data, data.length, server, port));
                byte[] buf = new byte[1024];
                DatagramPacket resp = new DatagramPacket(buf, buf.length);
                try {
                    socket.receive(resp);
                    System.out.println(new String(resp.getData(), 0, resp.getLength(),
                                                  StandardCharsets.UTF_8));
                } catch (SocketTimeoutException e) {
                    System.out.println("No reply within 2 s (datagram or reply lost?)");
                }
            }
        }
    }
}
```

7.2 Explanation

- There is **no connection and no accept()**: one server socket receives datagrams from all clients; the reply goes to the address and port found in the received packet.
- Each `send()` produces exactly one datagram and each `receive()` returns exactly one — **message boundaries are preserved**, so no framing is needed.
- Always use `getLength()`: the buffer may be larger than the datagram, and may still contain bytes from an earlier, longer message.
- If the buffer is smaller than the datagram, the extra bytes are **silently discarded**.
- The maximum UDP (User Datagram Protocol) payload over IPv4 (Internet Protocol version 4) is 65,507 bytes, but datagrams larger than about 1,472 bytes (Ethernet MTU (Maximum Transmission Unit) 1500 − 20 − 8) are fragmented by IP (Internet Protocol); losing one fragment loses the whole datagram. Keep datagrams small.
- **Reliability is the application's job:** the client uses a timeout; a robust client also retransmits a limited number of times and puts a request identifier in each message to match replies with requests.
- `DatagramSocket.connect(addr, port)` does not send anything; it only fixes the peer, filters datagrams from others and lets ICMP (Internet Control Message Protocol) errors surface as `PortUnreachableException`.

8. TCP (Transmission Control Protocol) vs UDP (User Datagram Protocol) programming; good practices

Table 10. *Comparison of TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) programming*

Aspect	TCP (Transmission Control Protocol) (stream sockets)	UDP (User Datagram Protocol) (datagram sockets)
Java classes	ServerSocket, Socket	DatagramSocket, DatagramPacket
Connection set-up	Yes: constructor of Socket / accept()	No
Sockets on the server	1 listening + 1 per client	1 for all clients
Data unit	Byte stream (needs framing)	Datagram (boundaries preserved)
Reliability, ordering	Guaranteed by TCP (Transmission Control Protocol)	Must be added by the application if needed
I/O (Input/Output) interface	InputStream / OutputStream	send() / receive() of packets

Aspect	TCP (Transmission Control Protocol) (stream sockets)	UDP (User Datagram Protocol) (datagram sockets)
End of communication	Detected: read returns -1 / null	Not detected: use timeouts or explicit messages
Broadcast / multicast	Not possible	Possible
Overhead and latency	Higher (handshake, ACKs (Acknowledgments), 20+ byte headers)	Lower (8-byte header, no handshake)

Table 11. *Typical errors in socket programs*

Symptom	Likely cause	Remedy
Client blocks forever after sending	Missing flush, or missing newline when the peer uses readLine()	Use println() with auto-flush, or call flush()
Second client waits until the first leaves	Iterative server	Serve each client in a thread / pool
BindException at server start	Port in use (other program or previous instance)	Stop the other process, choose another port, use setReuseAddress
Works on localhost, not between two PCs (Personal Computers)	Firewall blocks the port, or server bound to 127.0.0.1	Allow the port in the firewall; bind to 0.0.0.0 (default)
Garbage characters at the end of a UDP (User Datagram Protocol) message	Using getData() without getLength()	new String(p.getData(), 0, p.getLength(), UTF_8)
Accented characters broken between Windows and Linux	Default charset differs	Always specify UTF-8 (8-bit Unicode Transformation Format) explicitly
Server has many CLOSE_WAIT sockets	Server never closes accepted sockets	Use try-with-resources

9. Summary and key terms

- A socket is an OS (Operating System)-managed endpoint identified by (IP (Internet Protocol) address, port, protocol); applications use it through the socket API (Application Programming Interface).
- Sockets are classified by address family, type (stream, datagram, raw, seqpacket), role, I/O (Input/Output) model and delivery mode.
- TCP (Transmission Control Protocol) server: ServerSocket → accept → streams → close; TCP (Transmission Control Protocol) client: Socket(host, port) → streams → close.
- UDP (User Datagram Protocol) server and client both use DatagramSocket and DatagramPacket; there is no connection and the application handles loss.
- TCP (Transmission Control Protocol) needs framing (delimiters or length prefixes); UDP (User Datagram Protocol) preserves message boundaries.
- Concurrent servers use threads, thread pools, virtual threads or NIO (New Input/Output) selectors.

Key terms: socket, socket address, handle / file descriptor, Berkeley sockets, Winsock, address family, stream socket, datagram socket, raw socket, listening socket, connected socket, backlog, blocking / non-blocking I/O (Input/Output), iterative / concurrent server, framing, application protocol, timeout.

Part II — Seminar: analysis, debugging and protocol design

Problem 1 — Which call causes which packets?

For the TCP (Transmission Control Protocol) echo programs (Listings 2 and 3), state which Java call triggers each of the following and in which thread/process it happens: (a) the SYN (Synchronize) segment; (b) the moment `accept()` returns; (c) the first data segment; (d) the FIN (Finish) from the client; (e) an RST (Reset) instead of a SYN-ACK (Synchronize-Acknowledgment).

> **Solution**
>
> (a) `new Socket(host, port)` in the client (it calls connect()).
>
> (b) The server kernel completes the handshake by itself when the final ACK (Acknowledgment) arrives; `accept()` then dequeues the new connection. The handshake may even complete before `accept()` is called.
>
> (c) `out.println(line)` in the client (auto-flush pushes the bytes to TCP (Transmission Control Protocol)).
>
> (d) Closing the client socket at the end of the try-with-resources block (or `shutdownOutput()`).
>
> (e) The client connects to a port where no server listens; the client receives `ConnectException: Connection refused`.

Problem 2 — Counting sockets

The TCP (Transmission Control Protocol) echo server runs on `192.168.1.100:5000`. Three clients are connected: two from `192.168.1.21` (ports 50510 and 50511) and one from `192.168.1.35` (port 61200). (a) How many sockets does the server process have open? (b) Write the 5-tuple of each connection. (c) What do `netstat -an` (Windows) or `ss -tan` (Linux) show on the server for port 5000?

> **Solution**
>
> (a) **4 sockets**: 1 listening `ServerSocket` + 3 connected `Socket` objects. All of them have local port 5000.
>
> (b) (TCP (Transmission Control Protocol), 192.168.1.100, 5000, 192.168.1.21, 50510); (TCP (Transmission Control Protocol), 192.168.1.100, 5000, 192.168.1.21, 50511); (TCP (Transmission Control Protocol), 192.168.1.100, 5000, 192.168.1.35, 61200).
>
> (c) One line `0.0.0.0:5000 … LISTEN` and three lines `192.168.1.100:5000 <client> ESTABLISHED`.

Problem 3 — Find the bug

Each fragment below compiles but does not work correctly. Find the problem and correct it.

Fragment (a)

```
// (a) client
PrintWriter out = new PrintWriter(socket.getOutputStream());
out.println("HELLO");
String reply = in.readLine();          // blocks forever
```

Fragment (b)

```
// (b) server, the client uses readLine()
out.print("ECHO: " + line);           // out has auto-flush enabled
```

Fragment (c)

```
// (c) UDP (User Datagram Protocol) server
socket.receive(packet);
String msg = new String(packet.getData(), StandardCharsets.UTF_8);
```

Fragment (d)

```
// (d) UDP (User Datagram Protocol) server reusing one packet object
byte[] buf = new byte[1024];
DatagramPacket packet = new DatagramPacket(buf, buf.length);
while (true) {
    socket.receive(packet);           // after a 5-byte message, later messages are cut to 5 bytes
    handle(packet);
}
```

Fragment (e)

```
// (e) server
while (true) {
    Socket s = server.accept();
    serveUntilBye(s);                 // other clients wait
}
```

Solution

(a) No auto-flush: "HELLO" stays in the buffer. Use `new PrintWriter(socket.getOutputStream(), true)` or call `out.flush()`.

(b) `print` sends no newline, so `readLine()` on the client never completes; auto-flush works only with `println`. Use `out.println(...)`.

(c) `getData()` returns the whole 1024-byte buffer, including old or zero bytes. Use `new String(packet.getData(), 0, packet.getLength(), UTF_8)`.

(d) `receive()` sets the packet's length to the size of the datagram received; the next `receive()` can store at most that many bytes. Call `packet.setLength(buf.length)` before each `receive()`, or create a new packet in the loop.

(e) Iterative server: one client blocks the others. Serve each client in a separate thread or in a thread pool.

Problem 4 — What does the program see?

Table 12. *Error behaviour of TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) programs*

Situation	TCP (Transmission Control Protocol) client / server	UDP (User Datagram Protocol) client / server
Server not running (same host)	ConnectException: Connection refused (RST (Reset))	send() succeeds; ICMP (Internet Control Message Protocol) Port Unreachable is returned; an unconnected socket ignores it → receive() times out; a connected socket gets PortUnreachableException
Server host down or packets dropped by a firewall	connect() hangs, then times out (SYN (Synchronize) retransmissions)	receive() times out
Server started twice on the same port	BindException: Address already in use	BindException: Address already in use
Server killed during the session	Client: read returns null / -1 (FIN (Finish)) or SocketException: Connection RST (Reset)	Client: receive() times out

Discuss: why does a UDP (User Datagram Protocol) sender *not* learn about the loss of its datagram, while a TCP (Transmission Control Protocol) sender always knows (eventually) whether its data arrived?

Problem 5 — Designing an application protocol

Design a text protocol for a **calculator service** over TCP (Transmission Control Protocol). The client sends operations on two numbers; the server returns the result or an error. Then adapt the design for UDP (User Datagram Protocol).

> **A possible solution**
>
> **Requests** (one line each, ending in \n): `ADD a b`, `SUB a b`, `MUL a b`, `DIV a b`, `QUIT`.
>
> **Replies** (one line): `OK <result>` or `ERR <message>` — e.g. `ERR division by zero`, `ERR unknown operation POW`, `ERR operands must be numbers`.
>
> **Session:** after connecting the client may send any number of requests; `QUIT` is answered with `OK bye` and the server closes the connection.
>
> **UDP (User Datagram Protocol) version:** each request must be self-contained and carry an identifier, e.g. `17 ADD 3 4` → `17 OK 7`. The client retransmits after a timeout (at most 3 times) and ignores replies whose identifier does not match. Since ADD/SUB/MUL/DIV are *idempotent*, duplicate execution caused by retransmission is harmless.
>
> A reference implementation of the TCP (Transmission Control Protocol) server is given in the Appendix.

Problem 6 — Framing

A TCP (Transmission Control Protocol) client executes `out.write("HELLO".getBytes()); out.write("WORLD".getBytes());`. The server executes `int n = in.read(buf);` once. List the possible values of n and the content of buf. Why is the result not deterministic? Propose two ways to make the server receive exactly the two words.

> **Solution**
>
> n can be any value from 1 to 10: "HELLOWORLD" in one read, or "HELLO", or "HEL", etc. TCP (Transmission Control Protocol) delivers a stream; segment boundaries depend on Nagle's algorithm, buffer sizes, MSS (Maximum Segment Size) and timing, and the receiver's read returns whatever is available.
>
> Solutions: (1) a delimiter — send `HELLO\nWORLD\n` and read with `readLine()`; (2) a length prefix — `writeInt(5)` + bytes, read with `readInt()` + `readFully()` (Listing 5).

Proposed exercises (homework)

1. Rewrite the TCP (Transmission Control Protocol) echo server so that it is iterative. Run two clients and describe what happens.
2. Modify the TCP (Transmission Control Protocol) echo client to set a connect timeout of 3 s and a read timeout of 5 s. Test it with an unreachable address.
3. Write a UDP (User Datagram Protocol) client that sends 100 numbered datagrams as fast as possible and a server that counts them and reports missing numbers. Run it between two machines over Wi-Fi.
4. Explain why an HTTP/1.1 (Hypertext Transfer Protocol version 1.1) server must know the length of each response body (Content-Length or chunked encoding).
5. Compare the number of packets needed for one request-reply exchange with TCP (Transmission Control Protocol) (including connection set-up and release) and with UDP (User Datagram Protocol).

Part III — Laboratory

Both tracks use the **same Java programs** (Listings 2, 3, 6 and 7 from the lecture) and the same assignments; only the operating-system commands differ. Each track takes about 100 minutes, plus the programming assignments. Work in pairs when two computers are needed.

Table 13. *Map of laboratory tasks*

Task	Goal	Windows	Linux
0	Set up the JDK (Java Development Kit) and a working folder	winget, PowerShell	apt
1	Compile and run the TCP (Transmission Control Protocol) echo server and client	javac / java	javac / java
2	Observe the sockets of the server and its clients	netstat, Get-NetTCPConnection	ss, jcmd / jstack
3	Test the server with a generic client	ncat, Test-NetConnection	nc
4	Run client and server on two machines	ipconfig, New-NetFirewallRule	ip addr, ufw / firewalld
5	Compile and run the UDP (User Datagram Protocol) echo programs	Get-NetUDPEndpoint, ncat -u	ss -u, nc -u
6	Capture the traffic of both programs	Wireshark (Npcap loopback)	tcpdump
7	Provoke and explain the typical errors	—	privileged ports

Lab 7-W: Java sockets on Windows (Windows 10/11)

Task W0 — Setting up

PowerShell

```
winget install EclipseAdoptium.Temurin.21.JDK     # or download from adoptium.net
# open a NEW PowerShell window so that PATH is updated
java -version
javac -version
mkdir C:\cn\week07; cd C:\cn\week07
```

Save the four programs (TcpEchoServer.java, TcpEchoClient.java, UdpEchoServer.java, UdpEchoClient.java) in `C:\cn\week07`, using UTF-8 (8-bit Unicode Transformation Format) encoding. Any editor works (VS (Visual Studio) Code, IntelliJ IDEA, Notepad++). If `javac` is not recognised, add `C:\Program Files\Eclipse Adoptium\jdk-21…\bin` to the PATH.

Task W1 — Running the TCP (Transmission Control Protocol) echo server and client

Compile and run

```
# PowerShell window 1
javac TcpEchoServer.java TcpEchoClient.java
java TcpEchoServer 5000

# PowerShell window 2
java TcpEchoClient localhost 5000
hello sockets
ECHO: HELLO SOCKETS
BYE
```

> **Single-file launch (JDK (Java Development Kit) 11+)**
> For quick tests, a single class can be run without compiling first: `java TcpEchoServer.java 5000`.

When the server starts for the first time, **Windows Defender Firewall** may ask whether to allow Java on the network. For tests on one machine you can cancel; for Task W4 allow it **only on private networks**.

Open two more windows and start two more clients at the same time. Check in the server window that all clients are served in parallel (one thread each).

Task W2 — Observing the sockets

PowerShell (third window)

```
netstat -ano | findstr :5000
Get-NetTCPConnection -LocalPort 5000 | Format-Table
LocalAddress,LocalPort,RemoteAddress,RemotePort,State,OwningProcess
Get-NetTCPConnection -RemotePort 5000 | Format-Table LocalPort,State,OwningProcess
Get-Process -Id <pid>          # the java.exe process of the server
```

Questions W2

- How many lines with local port 5000 do you see with three clients connected? Relate this to Seminar Problem 2.
- Which ephemeral ports did the clients receive? Check that they belong to the Windows dynamic range.
- Stop one client with `BYE` and the server with Ctrl+C. Which side remains in TIME_WAIT in each case?

Task W3 — Testing with a generic client

PowerShell

```
Test-NetConnection localhost -Port 5000       # handshake only
ncat localhost 5000                            # type lines; the server answers (Nmap's ncat)
```

This shows an important property of protocols: **any** program that follows the protocol (here: lines of text over TCP (Transmission Control Protocol)) can talk to the server, regardless of the language it is written in.

Task W4 — Client and server on two machines

PowerShell

```
# On the SERVER machine (PowerShell as administrator)
ipconfig                                          # note the IPv4 (Internet Protocol version 4) address,
e.g. 192.168.10.21
New-NetFirewallRule -DisplayName "CN Lab port 5000 tcp" -Direction Inbound -Protocol TCP -LocalPort 5000
-Action Allow -Profile Private
New-NetFirewallRule -DisplayName "CN Lab port 6000 udp" -Direction Inbound -Protocol UDP -LocalPort 6000
-Action Allow -Profile Private
java TcpEchoServer 5000

# On the CLIENT machine
Test-NetConnection 192.168.10.21 -Port 5000
java TcpEchoClient 192.168.10.21 5000

# At the end, on the SERVER machine (administrator)
Remove-NetFirewallRule -DisplayName "CN Lab port 5000 tcp"
Remove-NetFirewallRule -DisplayName "CN Lab port 6000 udp"
```

If it does not work

- Check that the network profile is *Private* (`Get-NetConnectionProfile`).
- Check that both machines are in the same network (ping may be blocked by the firewall even when TCP (Transmission Control Protocol) works).
- The firewall prompt may have created a *Block* rule for java.exe: look for it with `Get-NetFirewallApplicationFilter | Where-Object Program -like '*java*' | Get-NetFirewallRule` or in *wf.msc*.

Task W5 — Running the UDP (User Datagram Protocol) echo programs

PowerShell

```
# Window 1
javac UdpEchoServer.java UdpEchoClient.java
java UdpEchoServer 6000

# Window 2
java UdpEchoClient localhost 6000

# Window 3
Get-NetUDPEndpoint -LocalPort 6000
ncat -u localhost 6000                    # generic UDP (User Datagram Protocol) client
```

Stop the server (Ctrl+C) and send another line from the client. Observe the message *No reply within 2 s*. Start the server again: the client continues working without reconnecting — there was never a connection.

Task W6 — Capturing the traffic

In Wireshark, select **Adapter for loopback traffic capture** (Npcap) and use the display filter `tcp.port == 5000 || udp.port == 6000`. Send one line with the TCP (Transmission Control Protocol) client and one with the UDP (User Datagram Protocol) client, then close both clients.

Questions W6

- Count the packets for one complete TCP (Transmission Control Protocol) session (connect, one line, BYE, close) and for one UDP (User Datagram Protocol) request-reply. Explain the difference.
- Find your text in the TCP (Transmission Control Protocol) payload (*Follow → TCP (Transmission Control Protocol) Stream*). Where are the message boundaries?
- Which segment corresponds to `new Socket(...)`, which to `println`, which to `close()`?

Task W7 — Provoking the typical errors

- Run `java TcpEchoClient localhost 5999` with no server → **ConnectException: Connection refused**.
- Start `java TcpEchoServer 5000` twice → the second fails with **BindException: Address already in use**.
- Run the client with a wrong host name → **UnknownHostException**.
- Kill the server with Ctrl+C while a client is connected, then type a line in the client → `null` reply or **Connection reset**.

Lab 7-L: Java sockets on Linux (Ubuntu 22.04/24.04 or Debian 12)

Task L0 — Setting up

Terminal

```
sudo apt update
sudo apt install -y openjdk-21-jdk netcat-openbsd tcpdump iproute2   # openjdk-17-jdk on older releases
java -version; javac -version
mkdir -p ~/cn/week07 && cd ~/cn/week07
```

Save the four Java programs in `~/cn/week07`. Use several terminals (or `tmux`) to run servers and clients at the same time.

Task L1 — Running the TCP (Transmission Control Protocol) echo server and client

Terminal

```
# Terminal 1
javac TcpEchoServer.java TcpEchoClient.java
java TcpEchoServer 5000

# Terminal 2
java TcpEchoClient localhost 5000

# Running the server in the background of a single terminal
java TcpEchoServer 5000 &      # 'jobs' lists it, 'kill %1' stops it
```

Task L2 — Observing the sockets and threads

Terminal

```
ss -tan '( sport = :5000 or dport = :5000 )'
ss -tlnp 'sport = :5000'              # listening socket and owning process
jcmd                                  # list Java processes and their PIDs (Process Identifiers)
jstack <pid> | grep '"Thread-'        # one thread per connected client
ps -o nlwp= -p <pid>                  # total number of threads of the JVM (Java Virtual Machine)
```

Questions L2

- Answer **Questions W2** for Linux.
- Connect three clients and verify with `jstack` that there are three client-handler threads. What happens to their number when the clients leave?

Task L3 — Testing with a generic client

Terminal

```
nc localhost 5000                                  # interactive
printf 'first line\nsecond line\nBYE\n' | nc -N localhost 5000   # scripted (-N: close after EOF (End Of File))
```

Task L4 — Client and server on two machines

Terminal

```
# On the SERVER machine
ip -4 addr show                          # note the address, e.g. 192.168.10.31
sudo ufw allow 5000/tcp; sudo ufw allow 6000/udp          # Ubuntu with ufw active
# or: sudo firewall-cmd --add-port=5000/tcp --add-port=6000/udp   (firewalld, not permanent)
java TcpEchoServer 5000

# On the CLIENT machine (Linux or Windows!)
nc -zv 192.168.10.31 5000
java TcpEchoClient 192.168.10.31 5000

# At the end, on the SERVER machine
sudo ufw delete allow 5000/tcp; sudo ufw delete allow 6000/udp
```

Cross-platform test

Run the server on Linux and the client on Windows (or the other way round). Because the programs specify UTF-8 (8-bit Unicode Transformation Format) explicitly and use a text protocol, they interoperate without any change — the network does not care about the operating system.

Task L5 — Running the UDP (User Datagram Protocol) echo programs

Terminal

```
# Terminal 1
javac UdpEchoServer.java UdpEchoClient.java
java UdpEchoServer 6000

# Terminal 2
java UdpEchoClient localhost 6000

# Terminal 3
ss -uanp 'sport = :6000'
echo "hello from nc" | nc -u -w1 localhost 6000
```

As on Windows, stop and restart the server while the client is running, and observe the timeouts.

Task L6 — Capturing the traffic

Terminal

```
sudo tcpdump -i lo -nn -A 'tcp port 5000 or udp port 6000'     # -A prints the payload as text
```

Questions L6

- Answer **Questions W6** using the tcpdump output (flags [S], [S.], [.], [P.], [F.]).
- Where do you see the text of your messages? Does each `println()` produce exactly one segment?

Task L7 — Typical errors, including privileged ports

- Reproduce the four errors from Task W7.
- Run `java TcpEchoServer 80` as a normal user → **BindException: Permission denied**: on Linux, ports below `net.ipv4.ip_unprivileged_port_start` (default 1024) are reserved for root. Windows has no such restriction. Do not run the server as root; use a port ≥ 1024.
- Start the server, connect a client, stop the server and immediately restart it. If you get *Address already in use*, check `ss -tan state time-wait` and explain the role of `setReuseAddress(true)`.

Programming assignments (both tracks)

Solve at least **A1 and A2**; A3 and A4 are for extra points. Test every program on your own machine and between two machines, and include the test commands and outputs in the report.

Table 14. *Programming assignments*

#	Assignment	Requirements
A1	TCP (Transmission Control Protocol) calculator server and client	Implement the protocol of Seminar Problem 5 (ADD, SUB, MUL, DIV, QUIT; replies OK / ERR). The server must be concurrent (thread pool) and must survive invalid input.
A2	UDP (User Datagram Protocol) time service with retransmission	The server answers the request `TIME` with the current date and time (`java.time.LocalDateTime.now()`). The client waits 1 s for a reply and retries at most 3 times before reporting failure. Test it while stopping the server.
A3	TCP (Transmission Control Protocol) file transfer	The client sends a file name and size, then the file content, using length-prefix framing (Listing 5). The server saves it in a folder `received/` and replies with `OK <bytes>`. Compare the received file with the original (fc /b on Windows, cmp on Linux).
A4	Multi-client chat (bonus)	Every line sent by a client is broadcast to all connected clients, prefixed by the sender's name. Keep the writers in a thread-safe collection (e.g. CopyOnWriteArrayList) and remove clients that disconnect.

Evaluation criteria

- Correct behaviour according to the protocol, including error cases (40 %).
- Robustness: timeouts, closing resources, no crash on invalid input or abrupt disconnects (25 %).
- Tests on one machine and on two machines, with the OS (Operating System) commands used for verification (20 %).
- Code clarity and comments (15 %).

Lab report template

Table 15. *Structure of the lab report*

Section	Content
1. Identification	Name, group, date, track (Windows / Linux), OS (Operating System) and JDK (Java Development Kit) versions
2. TCP (Transmission Control Protocol) echo	Commands and outputs for Tasks 1–3; socket listing with three clients; answers to Questions 2
3. Two machines	IP (Internet Protocol) addresses, firewall commands, proof of a successful remote session
4. UDP (User Datagram Protocol) echo	Outputs of Task 5; behaviour when the server is stopped
5. Traffic analysis	Capture (screenshot or text) and packet counts for TCP (Transmission Control Protocol) vs UDP (User Datagram Protocol); answers to Questions 6
6. Errors	Each provoked error with its cause
7. Assignments	Source code, test commands and outputs for A1, A2 (and A3, A4)

Appendix — Reference solution for A1 (instructor)

A concurrent calculator server using a thread pool. It can be tested with the TCP (Transmission Control Protocol) echo client from Listing 3 (`java TcpEchoClient localhost 5001`) or with `ncat` / `nc`.

Listing 8. CalcServer.java

```
import java.io.*;
import java.net.*;
import java.util.concurrent.*;

/** Protocol: request "OP a b\n" with OP in {ADD,SUB,MUL,DIV};
 *  reply "OK result\n" or "ERR message\n". */
public class CalcServer {
    public static void main(String[] args) throws IOException {
        int port = (args.length > 0) ? Integer.parseInt(args[0]) : 5001;
        ExecutorService pool = Executors.newFixedThreadPool(20);
        try (ServerSocket server = new ServerSocket(port)) {
            System.out.println("Calculator server on port " + port);
            while (true) {
                Socket c = server.accept();
                pool.submit(() -> serve(c));
            }
        }
    }

    static void serve(Socket c) {
        try (c;
             BufferedReader in = new BufferedReader(
                     new InputStreamReader(c.getInputStream(), "UTF-8"));
             PrintWriter out = new PrintWriter(
                     new OutputStreamWriter(c.getOutputStream(), "UTF-8"), true)) {
            String line;
            while ((line = in.readLine()) != null) {
                if (line.trim().equalsIgnoreCase("QUIT")) { out.println("OK bye"); break; }
                out.println(compute(line.trim()));
            }
        } catch (IOException e) { /* client vanished */ }
    }

    static String compute(String req) {
        String[] p = req.split("\\s+");
        if (p.length != 3) return "ERR usage: OP a b";
        try {
            double a = Double.parseDouble(p[1]), b = Double.parseDouble(p[2]);
            switch (p[0].toUpperCase()) {
                case "ADD": return "OK " + (a + b);
                case "SUB": return "OK " + (a - b);
                case "MUL": return "OK " + (a * b);
                case "DIV": return (b == 0) ? "ERR division by zero" : "OK " + (a / b);
                default:    return "ERR unknown operation " + p[0];
            }
        } catch (NumberFormatException e) {
            return "ERR operands must be numbers";
        }
    }
}
```

Sample session (verified)

```
ADD 3 4        ->  OK 7.0
DIV 1 0        ->  ERR division by zero
POW 2 3        ->  ERR unknown operation POW
MUL 2.5 4      ->  OK 10.0
QUIT           ->  OK bye
```

References

- J. F. Kurose, K. W. Ross — *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021 — Sections 2.7 (socket programming) and 3.
- E. R. Harold — *Java Network Programming*, 4th ed., O'Reilly, 2013.
- W. R. Stevens, B. Fenner, A. M. Rudoff — *Unix Network Programming, Vol. 1: The Sockets Networking API (Application Programming Interface)*, 3rd ed., Addison-Wesley, 2003.
- Oracle — Java SE (Standard Edition) API (Application Programming Interface) documentation for the packages java.net and java.nio.channels; The Java Tutorials: *All About Sockets* and *All About Datagrams*.
- Microsoft Learn — Windows Sockets 2 (Winsock); New-NetFirewallRule; Get-NetTCPConnection. Linux man pages: socket(7), tcp(7), udp(7), ss(8), nc(1).